

URLs and URIs

Both URL (Uniform Resource Locator) and URI (Uniform Resource Identifier) are related to identifying resources on the internet, but they have distinct roles.

A **URI** is a broad term that refers to anything that identifies a resource, whether it's on the web or not. It can be:

- **A URL:** tells where and how to access the resource.
- **A URN:** Uniquely identifies a resource but doesn't specify how to locate it.

Example URIs:

- <https://example.com/page.html> (a URL, because it includes location)
- `Urn:isbn:0451450523` (a URN, uniquely identifying a book by ISBN)

A URL is a specific type of URI that provides a way to locate a resource by specifying:

- **Protocol** (e.g., http, https, ftp)
- **Domain or IP Address** (e.g., example.com)
- **Path** (e.g., /page.html)
- **Optional parameters** (e.g. ?query=search)

Example URLs:

- <https://www.example.com/article?id=123>

Relative URL

A relative URL is a type of URL that does not include the full address (protocol + domain). It is used to reference resources within the same website or application.

Example: /images/photo.jpg

: It refers to photo.jpg inside the images folder but relies on the base URL.

Example with a Base URL:

- Base URL: <https://example.com>
- Relative URL: /blog/article.html
- Full URL: <https://example.com/blog/article.html>

The URL Class

The “URL class” in programming, mostly referring to the Java class “URL”, is a data structure that represents a URL, essentially a web address that points to a specific resource on the internet, allowing developers to easily access and manipulate different parts of a URL like the protocol, domain, and the file path with in their code. The URL class extends `java.lang.Object` and is a final class that cannot be subclassed.

Creating New URLs

The easiest way to create a URL object is from a String that represents the human-readable form of the URL address. This is typically the form that another person will use for a URL. In your Java program, you can use a String containing this text to create a URL object. We can create a URL object using the new URL (String url) constructor.

Example:

```
import java.net.URL;

public class URLExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("https://example.com:8080/path/page.html?query=test#section");
            System.out.println("URL: " + url);
        } catch (Exception e) {
            System.out.println("Invalid URL: " + e.getMessage());
        }
    }
}
```

Retrieving Data from a URL

The URL class provides several methods to extract components of a URL. Some of the methods are given below:

- `.getProtocol()`: extract the internet protocols (https).
- `.getHost()`: extracts the host or domain (example.com)
- `.getPort()`: extracts the port number or address (8080).
- `.getPath()`: extracts the path (/docs/index.html)
- `.getQuery()`: extracts the query string (name=value).

Take reference: <https://example.com:8080/docs/index.html?name=value>

Example:

```
import java.net.URL;

public class URLParts {

    public static void main(String[] args) throws Exception {

        URL url = new URL("https://example.com:8080/docs/index.html?name=value#section1");

        System.out.println("Protocol: " + url.getProtocol()); // https
        System.out.println("Host: " + url.getHost());        // example.com
        System.out.println("Port: " + url.getPort());         // 8080
        System.out.println("Path: " + url.getPath());         // /docs/index.html
        System.out.println("Query: " + url.getQuery());        // name=value
    }

}
```

Splitting a URL into Pieces

We can split a URL into its components using the URL class methods like `getAuthority()`, `getFile()`, `getUserInfo()`.

Example:

```
import java.net.URL;

public class URLSplit {

    public static void main(String[] args) throws Exception {

        URL url = new URL("https://example.com/resource?query=data#header");

        System.out.println("Authority: " + url.getAuthority()); // example.com
        System.out.println("File: " + url.getFile());           // /resource?query=data
        System.out.println("User Info: " + url.getUserInfo());  // null (not present in URL)
    }

}
```

```
}
```

Comparing URLs

Java allows URL comparison using **equals()** and **sameFile()**.

Example:

```
import java.net.URL;
```

```
public class URLComparison {  
    public static void main(String[] args) throws Exception {  
        URL url1 = new URL("https://example.com/page");  
        URL url2 = new URL("https://example.com/page");  
        URL url3 = new URL("https://example.com/page#section");  
  
        System.out.println("URL1 equals URL2: " + url1.equals(url2)); // True  
        System.out.println("URL1 equals URL3: " + url1.equals(url3)); // False (fragment  
differs)  
        System.out.println("Same file: " + url1.sameFile(url3));    // True (ignores fragment)  
    }  
}
```

Converting URL to String

To convert a URL object back to a string, we can use **toString()** or **toExternalForm()**.

Example:

```
import java.net.URL;
```

```
public class URLToString {  
    public static void main(String[] args) throws Exception {  
        URL url = new URL("https://example.com/index.html?query=test");  
        System.out.println("URL as String: " + url.toString());  
        System.out.println("External Form: " + url.toExternalForm());  
    }  
}
```

```
}
```

The URI Class

The `java.net.URI` class represents a Uniform Resource Identifier (URI), which is a generic way to identify a resource, unlike a URL, which includes a way to access it. URIs can be absolute (full identifier) or relative (partial identifier).

The `URL` class is useful for manipulating URIs, resolving relative URIs, and comparing URIs efficiently. The `URI` class defines methods and properties for handling URIs.

1. Constructing a URI: We can create a URI using its constructor by passing a valid URI string.

Example:

```
import java.net.URI;
```

```
public class URIExample {  
    public static void main(String[] args) {  
        try {  
            URI uri = new URI("https://example.com/path/to/page?query=value#section");  
            System.out.println("URI: " + uri);  
        } catch (Exception e) {  
            System.out.println("Invalid URI: " + e.getMessage());  
        }  
    }  
}
```

- Unlike URL, URI does not perform immediate network validation.

2. The parts of a URI: A URI consists of multiple components.

Example: `scheme://authority/path?query#fragment`

```
import java.net.URI;
```

```
public class URIParts {  
    public static void main(String[] args) throws Exception {
```

```

URI uri = new URI("https://example.com:8080/docs/index.html?name=value#section1");

System.out.println("Scheme: " + uri.getScheme());    // https
System.out.println("Authority: " + uri.getAuthority()); // example.com:8080
System.out.println("Host: " + uri.getHost());        // example.com
System.out.println("Port: " + uri.getPort());        // 8080
System.out.println("Path: " + uri.getPath());        // /docs/index.html
System.out.println("Query: " + uri.getQuery());      // name=value
System.out.println("Fragment: " + uri.getFragment()); // section1
}
}

```

3. Resolving Relative URIs: We can resolve a relative URI against an absolute base URI.

Example:

```

import java.net.URI;

public class URIResolution {
    public static void main(String[] args) throws Exception {
        URI baseUri = new URI("https://example.com/docs/");
        URI relativeUri = new URI("page.html");

        URI resolvedUri = baseUri.resolve(relativeUri);
        System.out.println("Resolved URI: " + resolvedUri);
        // Output: https://example.com/docs/page.html
    }
}

```

- **resolve()** combines the base URI with relative path.

4. Equality & Comparison of URIs: The URI class provides methods for checking equality and ordering.

Example:

```
import java.net.URI;
```

```
public class URIComparison {  
    public static void main(String[] args) throws Exception {  
        URI uri1 = new URI("https://example.com/page");  
        URI uri2 = new URI("https://example.com/page");  
        URI uri3 = new URI("https://example.com/page#section");  
  
        System.out.println("URI1 equals URI2: " + uri1.equals(uri2)); // True  
        System.out.println("URI1 equals URI3: " + uri1.equals(uri3)); // False (fragment differs)  
        System.out.println("URI1      and      URI3      normalized:      "      +  
uri1.normalize().equals(uri3.normalize())); // True  
    }  
}
```

- normalize() helps in comparing URIs after removing redundant components.

5. String Representation of a URI: We can convert a URI to a string or retrieve its raw components.

Example:

```
import java.net.URI;
```

```
public class URIToString {  
    public static void main(String[] args) throws Exception {  
        URI uri = new URI("https://example.com/path/to/page?query=value#section");
```

```
System.out.println("URI as String: " + uri.toString());  
System.out.println("Raw Path: " + uri.getRawPath());    // Encoded path  
System.out.println("Raw Query: " + uri.getRawQuery());  // Encoded query  
System.out.println("Raw Fragment: " + uri.getRawFragment()); // Encoded fragment  
}  
}
```

- **Raw methods** (`getRawPath()`, `getRawQuery()`, `getRawFragment()`) return the original encoded form.

x-www-form-urlencoded

When designing the web, one of the biggest challenges was dealing with the differences between operating systems.

- **File Naming Differences:** Some OSs allow spaces in filenames, while others don't.
- **Special Characters:** Certain symbols (e.g., #, &, ?, =, /) have special meanings in URLs but might be valid in file names on an OS.
- **Encoding Issues:** Some characters need to be encoded to ensure they are interpreted correctly across all systems and browsers.

For example, consider a filename on windows: C:\users\John Doe\Documents\report

- Windows allows spaces (“ ”) and # in filenames.

- However, in a URL, a # is a fragment identifier, meaning anything after it is ignored by the server.

- If we send <https://example.com/docs/report#1.pdf>, the browser will assume the file is report and #1.pdf is a fragment.

The x-www-form-urlencoded format solves this issue by encoding special characters into a percent-encoded format, ensuring safe transmission over the web.

Character	Issue in URL	Encoded Form
Space (“ ”)	Gets removed or misinterpreted	%20 or +
# (Fragment)	Treated as a fragment identifier	%23
& (Ampersand)	Separates key-value pairs in query parameters.	%26
? (Question mark)	Starts query parameters.	%3F
= (Equal Sign)	Separates key and values	%3D

Example: Encoding a Filename

Let's encode the filename report #1.pdf to make it safer for use in a URL.

```
import java.net.URLEncoder;
import java.nio.charset.StandardCharsets;

public class URLEncodingExample {
    public static void main(String[] args) {
        try {
            String filename = "report #1.pdf";
            String encodedFilename = URLEncoder.encode(filename,
StandardCharsets.UTF_8);

            System.out.println("Original: " + filename);
            System.out.println("Encoded: " + encodedFilename);
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Output:

Original: report #1.pdf

Encoded: report+%231.pdf

In the output, we can clearly see that # is encoded as %23 and space is encoded as +. Now, the safe URL to access this file would be:

<https://example.com/docs/report+%231.pdf>

Decoding the URL

If a browser or server receives the encoded URL, it needs to decode it back:

```
import java.net.URLDecoder;
```

```
import java.nio.charset.StandardCharsets;
```

```
public class URLDecodingExample {
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            String encodedFilename = "report+%231.pdf";
```

```
            String decodedFilename = URLDecoder.decode(encodedFilename,  
StandardCharsets.UTF_8);
```

```
            System.out.println("Encoded: " + encodedFilename);
```

```
            System.out.println("Decoded: " + decodedFilename);
```

```
        } catch (Exception e) {
```

```
            System.out.println("Error: " + e.getMessage());
```

```
        }
```

```
    }
```

```
}
```

Output:

Encoded: report+%231.pdf

Decoded: report #1.pdf

- **Now the filename is correctly restored**

Why decoding matters?

- **Ensures URL Safety:** Prevents misinterpretation of special characters.
- **Cross-Platform Compatibility:** Works across different operating systems.
- **Prevents Security Issues:** Proper encoding prevents injection attacks and parsing errors.
- **Correct URL Handling:** Prevents breaking URLs due to special characters.

The differences in how operating systems handle filenames led to the need for encoding special characters in URLs. The x-www-form-urlencoded format ensures that spaces, #, &, and other special characters do not break the URL and can be safely used across different systems.

Proxies

A proxy is an intermediate server that forwards requests between a client and the internet. A proxy server receives a request for a remote server from a client. The proxy server makes a request to the remote server and forwards the result back to the client. Java provides built-in support for proxies through **System Properties**, the **Proxy** class and the **ProxySelector** class.

Configuring Proxies Using System Properties

Java allows configuring proxies at the JVM level using system properties. This is useful when working with HTTP, HTTPS, and FTP connections.

Example:

```
import java.net.URL;

import java.net.URLConnection;

public class SystemProxyExample {

    public static void main(String[] args) {

        try {

            // Set Proxy System Properties

            System.setProperty("http.proxyHost", "proxy.example.com");

            System.setProperty("http.proxyPort", "8080");

            System.setProperty("https.proxyHost", "proxy.example.com");
```

```

System.setProperty("https.proxyPort", "8443");

// Open a connection using the proxy
URL url = new URL("http://example.com");
URLConnection connection = url.openConnection();
connection.connect();

System.out.println("Connected to " + url);
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Using the Proxy Class

The `java.net.Proxy` class allows configuring proxy settings per connection instead of system-wide settings.

Example: Using the proxy class for HTTP Connection

```

import java.net.*;

public class ProxyClassExample {
    public static void main(String[] args) {
        try {
            // Define Proxy Server and Port
            Proxy proxy = new Proxy(Proxy.Type.HTTP, new
            InetSocketAddress("proxy.example.com", 8080));

            // Open URL Connection via Proxy
            URL url = new URL("http://example.com");

```

```

        HttpURLConnection connection = (HttpURLConnection) url.openConnection(proxy);

        // Connect and Print Response Code
        connection.connect();

        System.out.println("Connected to " + url + " with response: " +
connection.getResponseCode());

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Proxy Types:

- Proxy.Type.HTTP: HTTP proxy for web traffic.
- Proxy.Type.SOCKS: SOCKS Proxy for general TCP Connections.
- Proxy.Type.DIRECT: No proxy (direct connection).

Using the proxySelector Class

The ProxySelector class provides a flexible way to dynamically choose the appropriate proxy for different URLs.

Example: Using ProxySelector to choose proxies

```

import java.net.*;
import java.util.List;

public class ProxySelectorExample {
    public static void main(String[] args) {
        try {
            // Get default ProxySelector

```

```

ProxySelector defaultSelector = ProxySelector.getDefault();

// Select Proxies for a given URL
URI uri = new URI("http://example.com");
List<Proxy> proxies = defaultSelector.select(uri);

// Print Selected Proxies
for (Proxy proxy : proxies) {
    System.out.println("Selected Proxy: " + proxy);
}
} catch (Exception e) {
    e.printStackTrace();
}
}
}

```

Summary:

- System Property : Sets system-wide proxy settings.
- Proxy class: Configures proxies per connection.
- ProxySelector: Dynamically selects proxies for URLs.

Communicating with Server-side Programs through GET

When communicating with a server-side program using GET method, we send data as query parameters in the URL. The server processes the request and returns a response.

Example: This example demonstrates how to send a GET request to a server-side script and read the response.

```

import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;

```

```
import java.net.URL;
import java.net.URLEncoder;

public class GetRequestExample {
    public static void main(String[] args) {
        try {
            // Base URL of the server-side program (Change this to an actual API)
            String baseUrl = "https://example.com/api";

            // Query parameters (encode them properly)
            String param1 = URLEncoder.encode("value1", "UTF-8");
            String param2 = URLEncoder.encode("value2", "UTF-8");

            // Construct the full URL with query parameters
            String fullUrl = baseUrl + "?param1=" + param1 + "&param2=" + param2;

            // Create a URL object
            URL url = new URL(fullUrl);

            // Open a connection
            HttpURLConnection connection = (HttpURLConnection) url.openConnection();
            connection.setRequestMethod("GET"); // Specify GET method

            // Optional: Set request headers
            connection.setRequestProperty("User-Agent", "Mozilla/5.0");

            // Get the response code
            int responseCode = connection.getResponseCode();
```

```

        System.out.println("Response Code: " + responseCode);

        // Read response data
        BufferedReader in = new BufferedReader(new
InputStreamReader(connection.getInputStream()));

        String inputLine;
        StringBuilder response = new StringBuilder();

        while ((inputLine = in.readLine()) != null) {
            response.append(inputLine);
        }
        in.close();

        // Print the response
        System.out.println("Response from server: " + response.toString());

    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Accessing Password-protected Sites in Java

Java's URL class can access websites that use **HTTP authentication** by providing a username and password. However, many sites use **cookie-based authentication** (e.g., login forms), which is harder to handle because it varies between websites. Implementing cookie authentication requires full HTML form and cookie support, similar to a web browser. Java simplifies **standard HTTP authentication**, but handling **nonstandard authentication methods** (like cookies) is more complex.

While dealing with password-protected websites, Java provides the Authenticator, PasswordAuthentication, and JPasswordField classes to handle authentication.

1. **The Authenticator Class:** Java's Authenticator class allows you to set up automatic authentication for HTTP requests that require a username and password.

Example:

MyAuthenticator.java

```
import java.net.*;

class MyAuthenticator extends Authenticator {
    @Override
    protected PasswordAuthentication getPasswordAuthentication() {
        return new PasswordAuthentication("user", "passwd".toCharArray());
    }
}
```

AuthTest.java

```
import java.net.*;

public class AuthTest {
    public static void main(String[] args) {
        try {
            // ✓ Set the default authenticator
            Authenticator.setDefault(new MyAuthenticator());

            // ✓ Access a password-protected resource
            URL url = new URL("https://httpbin.org/basic-auth/user/passwd");
            HttpURLConnection connection = (HttpURLConnection) url.openConnection();

            // ✓ Send request and get response
            int responseCode = connection.getResponseCode();
            System.out.println("Response Code: " + responseCode);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```

    }
}

```

2. **The PasswordAuthentication Class:** This class securely stores user credentials for authentication.

Example:

```
import java.net.PasswordAuthentication;
```

```

public class PasswordAuthExample {
    public static void main(String[] args) {
        PasswordAuthentication auth = new PasswordAuthentication("admin",
"secret123".toCharArray());

        System.out.println("Username: " + auth.getUserName());
        System.out.println("Password: " + new String(auth.getPassword())); // Converting
char[] to String
    }
}

```

3. **The JPasswordField Class:** If we need user input for password, we use JPasswordField in Swing GUI.

Example:

```
import javax.swing.*;
```

```

public class PasswordInputExample {
    public static void main(String[] args) {
        JPasswordField passwordField = new JPasswordField();
        int option = JOptionPane.showConfirmDialog(null, passwordField, "Enter
Password", JOptionPane.OK_CANCEL_OPTION);

        if (option == JOptionPane.OK_OPTION) {
            char[] password = passwordField.getPassword();
            System.out.println("Password entered: " + new String(password)); // Not
recommended for real applications
        }
    }
}

```

How it Works?

- JPasswordField hides the typed password (*****).
- getPassword() returns a char array for better security.

- JOptionpane shows a popup to enter the password.